ZAID

Single Final Build Prompt

Futuristic dashboard + local Ollama chat integration

For Google Antigravity

This document combines the dashboard specification and local Ollama chat/backend specification into one integrated build request. It explicitly tells Antigravity to read both supplied specification PDFs before editing.

Project root: D:\AI\ZAID

Model: qwen3.5:4b via local Ollama

Scope: Integrated dashboard and local streaming chat only. No computer automation, phone controls, voice capture, external messaging, or further milestones.

IMPORTANT DOCUMENT INSTRUCTIONS

Before doing any work, read BOTH supplied project specification PDFs in full, including all pages, diagrams, images, checklists, and embedded prompt text:

1. `ZAID_Milestone1_Antigravity_Prompt.pdf` — dashboard visual design, routes, components, demo-data honesty, accessibility, responsive behavior, and UI tests.

2. The supplied ZAID local-Ollama chat/backend specification PDF — FastAPI architecture, Ollama integration, streaming, cancellation, security, and testing.

If either PDF is missing or inaccessible, explicitly report which one could not be read. Do not pretend to have read it. Use the detailed requirements in this prompt as the implementation baseline, and reconcile the two documents into ONE final application. Do not ask me to run separate milestone prompts. This is the single final build scope for the current project.

1. ROLE AND FINAL OBJECTIVE

Act as a senior full-stack engineer, UI engineer, Python/FastAPI developer, React/TypeScript developer, local-LLM integration engineer, accessibility reviewer, and application security reviewer.

Build ZAID, a Windows-first personal AI assistant interface with:

• A polished futuristic HUD dashboard inspired by the supplied reference image.

• A functional React/TypeScript frontend.

• A local FastAPI backend.

• A real chat connection to the user's local Ollama model `qwen3.5:4b`.

• Streaming assistant responses and working cancellation.

• Honest status indicators, safe errors, automated tests, and complete setup documentation.

The name of the assistant is ZAID. Never call it JARVIS.

This prompt replaces separate staged implementation prompts for the requested scope. Deliver the dashboard and local chat together as one integrated application. However, do NOT expand into unrelated autonomous computer control or phone features.

2. PROJECT PATH AND ABSOLUTE SAFETY RULES

Windows project root: `D:\AI\ZAID`

Frontend: `D:\AI\ZAID\frontend`

Backend: `D:\AI\ZAID\backend`

Existing separate project to protect: `D:\AI\OpenJarvis`.

Hard rules:

1. Work only inside `D:\AI\ZAID`. Do not create or modify project files elsewhere.

2. Never read, modify, move, overwrite, or delete anything inside `D:\AI\OpenJarvis`.

3. Before editing, inspect `D:\AI\ZAID`, report what exists, and read its existing README, package files, routes, components, styles, tests, Git status, and backend files if present.

4. If any file that would be changed contains existing user work that could be lost, stop and ask before overwriting it. Do not delete project files to simplify the task.

5. Do not run as administrator. Do not change global system settings or environment variables.

6. Do not install Qwen Code, agent frameworks, unrelated global packages, or another model.

7. Do not create a public service, port-forward, tunnel, remote Ollama endpoint, or network-exposed listener.

8. Do not upload project code or user data to external services.

9. Do not claim tests or integrations work unless actually tested and observed.

10. If the existing Milestone 1 dashboard is present, extend it instead of recreating it. Preserve user work and the existing visual identity.

11. If the project does not match the expected layout, adapt carefully and document deviations rather than destructively forcing the expected structure.

3. USER ENVIRONMENT AND MODEL

• OS: Windows.

• RAM: 16 GB.

• Tools previously installed: Python, Node.js, Git, Google Antigravity.

• Ollama executable was previously found at `C:\Users\thouf\AppData\Local\Programs\Ollama\ollama.exe`.

• Previously reported Ollama version: `0.40.5`; check the current actual version and API behavior.

• Model previously downloaded: `qwen3.5:4b`, reported size about 3.3 GB.

• Ollama endpoint expected: `http://127.0.0.1:11434`.

• Intended model directory: `D:\AI\OllamaModels`, but physical storage location is not confirmed.

• Do not inspect, move, or modify model files directly. Use Ollama's API.

• Do not download another model. If `qwen3.5:4b` is missing, report it and give the user a manual command rather than downloading automatically.

Before implementation, run safe read-only diagnostics:

• `python --version`

• `python -m pip --version`

• `node --version`

• `npm --version`

• `ollama --version`

• `ollama list`

• `ollama show qwen3.5:4b`

• `curl.exe http://127.0.0.1:11434/api/version`

If the local API cannot be reached, still build and test the application with mocked tests, but clearly report that live model verification was not possible.

4. REQUIRED ARCHITECTURE

Use:

• Frontend: React + TypeScript + Vite + React Router (reuse current project versions/configuration where possible).

• Backend: Python + FastAPI + Uvicorn.

• Ollama: local model API at `http://127.0.0.1:11434`.

• HTTP client: a maintained Python async HTTP client compatible with the installed Python version.

• Tests: existing project test setup if present; otherwise use Vitest/Testing Library for frontend and pytest/FastAPI TestClient or equivalent for backend.

• No database or persistent conversation storage in this scope.

• Chat state remains in React memory and resets on page reload.

Architecture:

React Chat UI -> relative `/api/*` routes through Vite proxy -> FastAPI bound to `127.0.0.1:8000` -> Ollama bound to `127.0.0.1:11434` -> `qwen3.5:4b`.

The frontend must never call Ollama directly. Do not hard-code Ollama's URL into browser code. The Vite development server must bind to `127.0.0.1`, and proxy `/api` to `http://127.0.0.1:8000`. FastAPI/Uvicorn must bind to `127.0.0.1`, never `0.0.0.0`.

Do not add wildcard CORS. Prefer the Vite proxy and narrow Host/Origin checks appropriate to local development. Document any residual local-development risks.

5. DESIGN AND BRANDING

Use the supplied ZAID HUD image as visual inspiration, not as an image to embed and not as a pixel-perfect copy.

Visual style:

• Near-black backgrounds.

• Neon red, deep red, orange, amber, and gold accents.

• Sparse cyan accent for selected data details.

• Central glowing orbital AI core.

• Technical typography, subtle circuit/grid patterns, corner brackets, restrained scanlines, radial gauges, sparklines, and glowing panel borders.

• Left quick-launch rail.

• Right integrations rail.

• System status, model status, storage, network, active tasks, events, recent activity, and voice panel.

• Central navigation dock for Chat, Computer, Files, Phone, Messages, Automation, Memory, Settings.

• Strong readability, responsive layout, visible keyboard focus, and reduced-motion support.

Keep existing CSS tokens and component system if Milestone 1 already created them. Avoid unnecessary dependencies. Do not add UI kits, CSS frameworks, animation libraries, chart libraries, icon packs, remote fonts, or CDN assets unless already present and necessary; ask before introducing a new category of dependency. Prefer inline SVG and CSS for icons, gauges, charts, and the core orb.

Use the exact product name ZAID. Set the document title to `ZAID - Personal AI System`.

6. HONEST STATUS AND DEMO DATA

Do not fake live system data or integrations.

• Any fabricated CPU, RAM, temperature, battery, storage, weather, network, task, event, or activity value must be sourced from a clearly named demo-data module and visibly labelled `DEMO DATA`.

• System metrics must not be collected from Windows in this scope.

• Weather must not call an API or use geolocation.

• Network values must not be detected from the machine.

• Local AI model status must be based on the actual backend status endpoint. Do not show ONLINE/CONNECTED unless verified.

• The model name `qwen3.5:4b` may be shown as the configured model; its availability must be checked.

• Do not display invented GPU/RAM/token-speed/latency values.

• Integrations remain `Unimplemented` unless a real integration exists; clicking their cards must never falsely connect them.

• Voice panel must say voice input is not implemented; microphone controls must remain disabled. Never request microphone or camera access.

• No phone calls, SMS, WhatsApp, email, or social-media integration in this scope.

• If tasks/events/activity are samples, label them as demo data. No false “running”, “secure”, “private”, “listening”, or “connected” claims.

• Decorative globe/core are visual elements only and must not imply a real secure connection.

7. DASHBOARD PAGES AND NAVIGATION

Preserve or implement these routes:

• `/` Overview

• `/chat` Chat

• `/computer`

• `/files`

• `/browser`

• `/phone`

• `/messages`

• `/calendar`

• `/integrations`

• `/automation`

• `/memory`

• `/activity`

• `/settings`

• Friendly 404 page

Menu drawer must list the 13 main pages in this order: Overview, Chat, Computer, Files, Browser, Phone, Messages, Calendar, Integrations, Automation, Memory, Activity, Settings. Make it keyboard accessible, close with Escape, restore focus, and identify the current page with `aria-current="page"`.

Overview should retain the full HUD layout:

1. Header with local browser clock/date, ZAID wordmark, tagline, decorative world graphic, and demo weather.

2. Left Quick Launch rail.

3. System Status panel with four demo radial gauges and a demo sparkline.

4. Local AI Model panel.

5. Storage panel with demo bars for Windows (C:), Data (D:), and Ollama Models (D:).

6. Central CoreOrb with caption `UI PREVIEW - AI CORE NOT CONNECTED` or similarly honest copy.

7. Network panel with demo-only values.

8. Active Tasks and Upcoming Events panels labelled demo.

9. Integrations rail with unimplemented statuses.

10. Bottom dock with navigation; central microphone button disabled.

11. Voice Input panel with static waveform, `Mic: off`, and disabled controls.

12. Recent Activity demo panel and decorative globe with caption that the preview does not make network requests only if this refers specifically to decorative/non-API dashboard features; do not imply the whole application makes no network requests once local chat is integrated.

The local browser clock is allowed. Clean up its interval on unmount.

Responsive layout:

• >=1280px: full HUD grid.

• 768–1279px: compact two-column layout.

• 360–767px: single-column stacked layout, touch targets at least 44px.

• Support widths through 1920px without horizontal page scrolling; at 2560px keep a centred max-width.

• Use CSS Grid/Flex, rem/clamp, and avoid rigid fixed-size composition.

Every non-Overview page except Chat should have a title, a clear description, an `Unimplemented` status, a planned-features list, and a note describing the future safety model. No non-working control should look functional.

Chat is the exception: it must work with the local Ollama model.

8. ACCESSIBILITY AND PERFORMANCE

• Semantic header/nav/main/aside landmarks.

• Skip-to-content link.

• One `h1` per page and logical heading hierarchy.

• Visible keyboard focus (at least 2px).

• Keyboard operation for navigation, drawer, chat composer, and buttons.

• Accessible names for icon-only buttons.

• Appropriate `aria-live` for notices; avoid announcing every token.

• Text contrast target at least 4.5:1.

• Honor `prefers-reduced-motion`; disable rotation, pulse, scanline, and smooth scrolling where appropriate.

• Pause decorative animation while the document is hidden if feasible.

• Animate primarily transform/opacity; avoid high-cost continuous effects.

• No `requestAnimationFrame` loop for decoration.

• Provide a top-level ErrorBoundary and friendly error state.

• Safe text rendering; never interpret model output as executable HTML.

9. BACKEND FILES AND CONFIGURATION

Create or extend a backend under `D:\AI\ZAID\backend`, using a local virtual environment `backend\.venv`. Do not install Python packages globally.

A suitable structure is:

backend/

app/

__init__.py

main.py

config.py

schemas.py

services/

__init__.py

ollama_service.py

tests/

requirements.txt

.env.example

.gitignore

README.md

Adapt to existing files if appropriate. Pin compatible dependencies to tested versions. Do not introduce a database.

Configuration defaults:

• Ollama URL: `http://127.0.0.1:11434`

• Model: `qwen3.5:4b`

• Backend host: `127.0.0.1`

• Backend port: `8000`

• Sensible request timeout.

• Maximum request body size.

• Maximum user message length.

• Maximum conversation message count.

No cloud API key or external provider is required.

10. BACKEND API

Implement:

`GET /api/health`

Reports only whether the ZAID backend is responding. It must not imply that Ollama is ready.

`GET /api/status`

Checks Ollama through an appropriate endpoint such as `/api/tags` and confirms whether `qwen3.5:4b` is available. Return structured fields such as:

• `backend_status`

• `ollama_status`

• `model_name`

• `model_available`

• `message`

Represent `available`, `disconnected`, `model_missing`, and `error` honestly. Do not expose environment variables, secrets, or unnecessary local paths.

`POST /api/chat/stream`

• Use Ollama's local `/api/chat` endpoint with streaming enabled.

• Accept a validated list of `user` and `assistant` messages only.

• Reject empty messages, invalid roles, excessive message length, excessive history, and oversized requests.

• Do not accept model names, Ollama URLs, system messages, or tool definitions from the browser.

• Construct a fixed server-side system prompt for ZAID. Explain that this application currently cannot access the user's computer, files, phone, or internet and must not claim to have done so.

• Use the configured model only.

• Stream response tokens.

• Handle Ollama failures and timeouts safely.

• On client disconnect/cancellation, close the Ollama HTTP stream and release resources.

• Do not log complete prompts or responses.

• Never execute model output as code, shell commands, or tools.

Use SSE with `Content-Type: text/event-stream`, or a comparably reliable documented stream format. If using SSE, define JSON events:

• `start`

• `token` with a text fragment

• `done`

• `error` with a safe user-facing message

Do not mark a disconnected stream as successful completion. Use Ollama completion metadata where available. Adapt to the installed Ollama API version and document any compatibility issue.

Security:

• Loopback binding only.

• No wildcard CORS.

• Narrow Host/Origin policy compatible with the local Vite proxy.

• Validated request schema and sensible timeouts.

• No arbitrary URL proxy.

• No shell endpoint or model tool calling.

• No secrets in logs.

• Safe errors without stack traces.

11. FRONTEND CHAT FUNCTIONALITY

Create or adapt a typed API client using relative URLs only:

• `/api/health`

• `/api/status`

• `/api/chat/stream`

The browser must never call `http://127.0.0.1:11434` directly.

Chat UI must support:

• Initial empty state.

• Checking connection.

• Backend unavailable.

• Ollama unavailable.

• Model missing.

• Ready to chat.

• Sending.

• Streaming.

• Completed response.

• Error.

• Cancelled generation.

The Chat header shows `qwen3.5:4b` and its actual status. Never show fake connected status or invented resource metrics.

Behavior:

• Enter sends.

• Shift+Enter inserts a newline.

• Empty messages cannot be sent.

• User message appears immediately.

• Assistant output streams in progressively.

• No fabricated AI responses.

• Keep conversation state in React memory only; no localStorage, IndexedDB, or database.

• Use safe text rendering.

• Display helpful error messages.

• Include a Stop button during generation.

• Use `AbortController` or equivalent cancellation.

• Cancellation must abort the browser request, trigger backend cleanup, stop loading state, mark the response cancelled, and permit another request.

• Do not add Markdown support unless it already exists and is safely implemented; do not add a new Markdown dependency without asking.

12. RUN COMMANDS AND DOCUMENTATION

Document the exact commands based on actual detected versions.

Backend terminal:

```
cd D:\AI\ZAID\backend
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
pip install -r requirements.txt
uvicorn app.main:app --host 127.0.0.1 --port 8000
```

Reuse an existing environment safely if present. Do not change the global PowerShell execution policy. If activation is blocked, document how to invoke the virtual environment's Python directly.

Frontend terminal:

```
cd D:\AI\ZAID\frontend
npm install
npm run dev
```

Use the Vite URL shown in the terminal, bound to `127.0.0.1`.

Document how to stop each process with Ctrl+C. Do not create Windows services or automatic startup tasks.

The root README must preserve existing Milestone 1 information and explain:

• Architecture.

• What works now and what remains unimplemented.

• Actual Python, Node, npm, and Ollama versions detected.

• Installation and startup commands.

• API endpoint checks.

• Tests.

• Troubleshooting.

• Security limitations.

• No persistent conversation storage.

13. GIT AND SECRET HYGIENE

Preserve the existing `.gitignore`; add necessary entries without removing existing rules. Ignore:

• `node_modules/`

• `dist/`

• `coverage/`

• `backend/.venv/`

• `__pycache__/`

• `.env` and secret-bearing environment files (allow `.env.example`)

• logs and databases

• model files and model folders

Do not reset Git, force-push, create a remote repository, or upload code. Report Git status if initialized.

14. TESTS AND VERIFICATION

Write and run tests.

Backend tests:

1. Health endpoint.
2. Status with Ollama available, unavailable, and model missing.
3. Reject invalid roles and empty messages.
4. Reject oversized messages and excessive history.
5. Client cannot override model or Ollama URL.
6. Streaming events are valid and ordered.
7. Ollama errors become safe error events.
8. Cancellation closes the upstream stream.
9. Unit tests use mocks and do not require a live model.

Frontend tests:

1. Status display reflects API result.
2. Backend failure never creates a fake assistant response.
3. Sending uses the expected relative endpoint.
4. Tokens stream into the current assistant message.
5. Enter and Shift+Enter behavior.
6. Empty message rejection.
7. Stop cancels generation.
8. Errors appear accessibly.
9. Sending works again after cancellation/error.
10. Navigation and placeholder pages still render.
11. Dashboard design tokens and status honesty remain intact.
12. No browser request goes directly to Ollama.

Run available checks:

• Python compilation.

• Backend unit tests.

• `npm run typecheck`

• `npm run lint`

• `npm run test`

• `npm run build`

If a script does not exist, add it only if appropriate to the existing setup. Fix issues introduced by this implementation. Do not claim a test passed without running it.

If feasible, start both servers and verify:

• `GET http://127.0.0.1:8000/api/health`

• `GET http://127.0.0.1:8000/api/status`

• A real chat message produces streamed text from `qwen3.5:4b`.

• Stop Generation actually cancels.

If the live model is unavailable, report that integration is unverified and provide exact manual checks.

15. MANUAL ACCEPTANCE CHECKLIST

Before declaring completion, verify:

• All changes are inside `D:\AI\ZAID`.

• `D:\AI\OpenJarvis` is untouched.

• Existing dashboard and all routes remain functional.

• HUD design, responsive behavior, and keyboard accessibility remain intact.

• All demo data is clearly labelled.

• Integrations remain unimplemented and do not claim connection.

• Microphone stays disabled and no permission prompt appears.

• Chat status reflects real backend/model status.

• A real prompt produces a real streamed response when Ollama is available.

• Stop Generation works and a new request can be submitted afterward.

• Frontend does not call Ollama directly.

• Both servers bind only to loopback.

• No secrets, model files, virtual environment, or build output are committed.

• Build and test results are recorded honestly.

16. FINAL REPORT REQUIRED

Provide:

1. Summary of the completed application.

2. Exact files created and modified.

3. Dependencies added and why.

4. Exact commands executed and outcomes.

5. Frontend test results.

6. Backend test results.

7. Build/typecheck/lint results.

8. Whether Ollama responded.

9. Whether `qwen3.5:4b` was verified.

10. Whether a real streamed response was tested.

11. Whether cancellation was verified.

12. Any checks that could not be performed.

13. Known limitations and deviations.

14. Exact commands to start ZAID next time.

Never fabricate results. If any required file needs destructive overwriting, stop and ask first.

17. FINAL SCOPE — DO NOT CREATE MORE MILESTONES

This is the last milestone in this requested build prompt. Do not create or propose another milestone as part of the implementation. Do not continue into computer control, file operations, persistent memory, voice, phone features, third-party integrations, MCP, cloud routing, or autonomous task execution. Those features are out of scope, not future tasks to start now.

Complete the integrated dashboard and local Ollama chat, test it, document it, and stop.